

UIDAI Data Hackathon 2026 - Submission Document



MBU Gap Analyzer

Predictive Gap Analysis Engine for Mandatory Biometric Updates

Team OMEGA

Table of Contents

- 1. [Problem Statement and Approach](#)
- 2. [Datasets Used](#)
- 3. [Methodology](#)
- 4. [Data Analysis and Visualisation](#)
- 5. [Code Files](#)

1. Problem Statement and Approach

1.1 The Problem: Children Losing Benefits Due to MBU Non-Compliance

Children enrolled in Aadhaar are **mandated** to update their biometrics at two critical life stages:

Age	Update Type	Reason
5 years	Mandatory Biometric Update (MBU)	Fingerprints mature, facial features change
15 years	Mandatory Biometric Update (MBU)	Adolescent biometric changes

What Happens If They Don't Update?

When children miss their Mandatory Biometric Updates, their Aadhaar becomes **INACTIVE**, leading to:

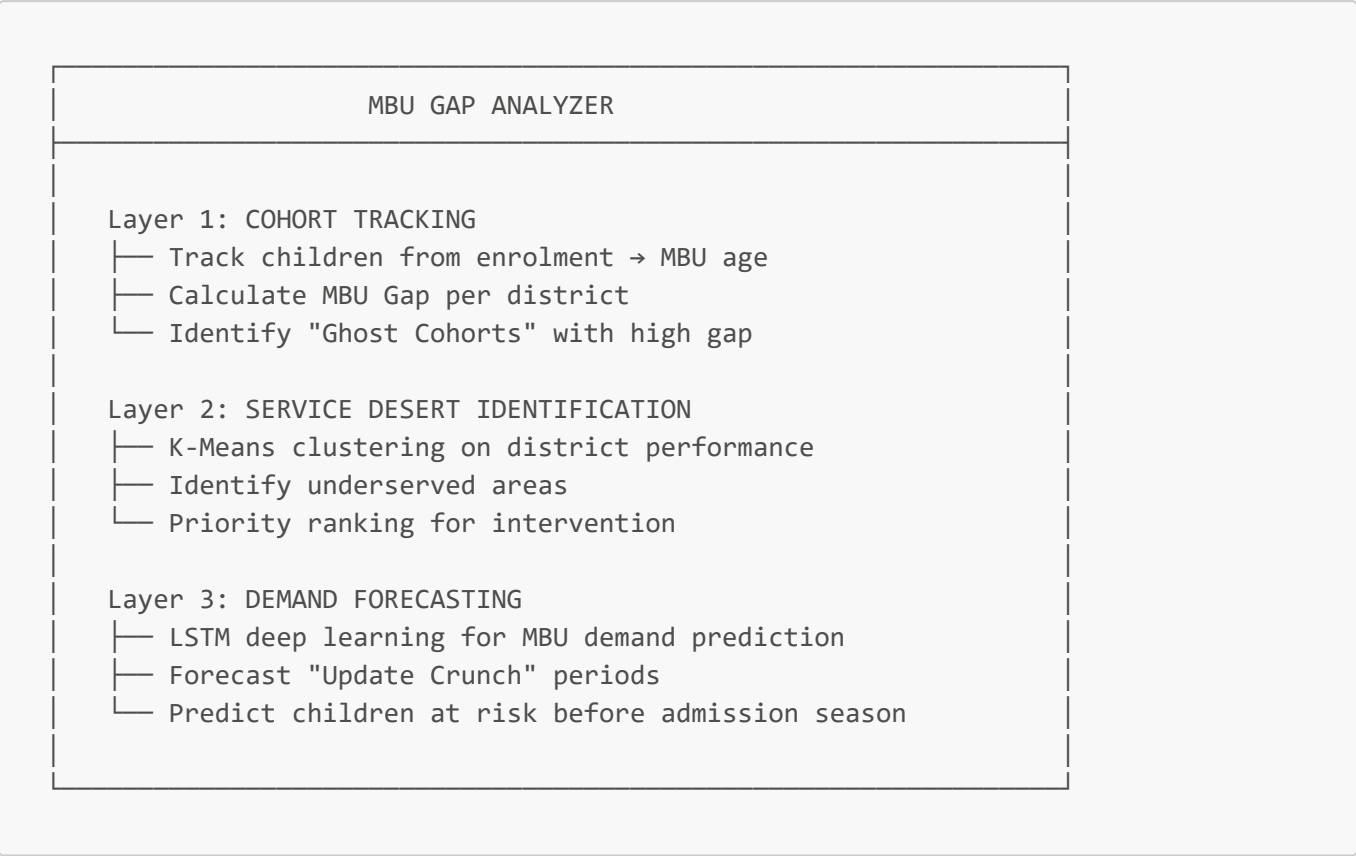
- ✗ School admission blocked
- ✗ Scholarship disbursement fails
- ✗ Mid-day meal authentication fails
- ✗ DBT (Direct Benefit Transfer) denied

The Scale of the Problem

Thousands of children risk losing government benefits worth crores of rupees annually because their Aadhaar wasn't updated on time — often due to lack of awareness or inaccessible update centers.

1.2 Our Solution: MBU Gap Analyzer

We built an intelligent **3-Layer Predictive Engine** that identifies **"Ghost Cohorts"** — children who were enrolled in Aadhaar but never completed their Mandatory Biometric Updates.



1.3 Technical Approach

We employ a hybrid **Machine Learning + Deep Learning** approach:

Component	Technology	Purpose
Data Processing	Pandas, NumPy	Clean and aggregate 5M+ records
Clustering	K-Means (Scikit-Learn)	Identify Service Desert districts
Time-Series	LSTM (PyTorch)	Forecast MBU demand
Anomaly Detection	Autoencoder (PyTorch)	Detect suspicious districts
Explainability	SHAP	Explain why districts are flagged
Visualization	Plotly, Folium	Interactive charts and maps

2. Datasets Used

2.1 Dataset Overview

We used **three official UIDAI datasets** provided for the hackathon:

Dataset	Records	Description
Aadhaar Enrolment	1,006,029	Children enrolled in Aadhaar by age group
Biometric Updates	1,861,108	Biometric update transactions
Demographic Updates	2,071,700	Demographic update transactions
Total	4,938,837	Combined records for analysis

2.2 Column Descriptions

Enrolment Dataset ([api_data_aadhar_enrolment](#))

Column	Data Type	Description
date	Date	Enrolment date (format: DD-MM-YYYY)
state	String	State name
district	String	District name
pincode	Integer	Postal code
age_0_5	Integer	Enrolments for age 0-5 years
age_5_17	Integer	Enrolments for age 5-17 years
age_18_greater	Integer	Enrolments for age 18+ years

Biometric Update Dataset ([api_data_aadhar_biometric](#))

Column	Data Type	Description
date	Date	Update date (format: DD-MM-YYYY)
state	String	State name
district	String	District name
pincode	Integer	Postal code
bio_age_5_17	Integer	Biometric updates for age 5-17 (MBU target group)
bio_age_17_	Integer	Biometric updates for age 17+

Demographic Update Dataset ([api_data_aadhar_demographic](#))

Column	Data Type	Description
date	Date	Update date (format: DD-MM-YYYY)
state	String	State name
district	String	District name
pincode	Integer	Postal code

Column	Data Type	Description
demo_age_5_17	Integer	Demographic updates for age 5-17
demo_age_17_	Integer	Demographic updates for age 17+

2.3 Key Computed Metrics

We derived the following metrics for gap analysis:

Metric	Formula	Purpose
MBU Eligible	age_0_5 + age_5_17 (enrolments)	Children who need MBU
MBU Gap	MBU Eligible - bio_age_5_17	Children who missed MBU
Compliance Ratio	(bio_age_5_17 / MBU Eligible) × 100	Update completion %

3. Methodology

3.1 Data Loading and Preprocessing

Step 1: Load CSV Files

```
def load_all_csvs(folder_path, dataset_name):
    """Load and concatenate all CSV files from a folder."""
    all_files = glob.glob(os.path.join(folder_path, "*.csv"))
    dfs = []
    for file in all_files:
        df = pd.read_csv(file)
        dfs.append(df)
    combined = pd.concat(dfs, ignore_index=True)
    return combined

# Load all datasets
df_enrolment = load_all_csvs("uidai_datasets/api_data_aadhar_enrolment",
                              "Enrolment")
df_biometric = load_all_csvs("uidai_datasets/api_data_aadhar_biometric",
                              "Biometric")
df_demographic = load_all_csvs("uidai_datasets/api_data_aadhar_demographic",
                                "Demographic")
```

Step 2: Date Parsing and Feature Engineering

```
def preprocess_dataset(df, name):
    if 'date' in df.columns:
        df['date'] = pd.to_datetime(df['date'], format='%d-%m-%Y',
                                     errors='coerce')
```

```
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['year_month'] = df['date'].dt.to_period('M')
return df
```

Step 3: Aggregation by District

```
# Aggregate enrolments by district
enrolment_by_district = df_enrolment.groupby(['state', 'district']).agg({
    'age_0_5': 'sum',
    'age_5_17': 'sum',
    'age_18_greater': 'sum'
}).reset_index()

# Calculate MBU Eligible
enrolment_by_district['mbu_eligible'] = (
    enrolment_by_district['age_0_5'] +
    enrolment_by_district['age_5_17']
)
```

3.2 Gap Analysis Computation

MBU Gap Calculation

```
# Merge enrolment and biometric data
df_gap_analysis = pd.merge(
    enrolment_by_district,
    biometric_by_district,
    on=['state', 'district'],
    how='outer'
).fillna(0)

# Calculate MBU Gap (children at risk)
df_gap_analysis['mbu_gap'] = (
    df_gap_analysis['mbu_eligible'] - df_gap_analysis['bio_age_5_17']
).clip(lower=0)

# Calculate Compliance Ratio
df_gap_analysis['mbu_compliance_ratio'] = np.where(
    df_gap_analysis['mbu_eligible'] > 0,
    (df_gap_analysis['bio_age_5_17'] / df_gap_analysis['mbu_eligible']) * 100,
    0
).clip(0, 100)
```

Risk Classification

```
def classify_risk(ratio):  
    if ratio >= 80:  
        return 'Green (Compliant)'  
    elif ratio >= 50:  
        return 'Yellow (Moderate Risk)'  
    else:  
        return 'Red (Service Desert)'  
  
df_gap_analysis['risk_category'] =  
df_gap_analysis['mbu_compliance_ratio'].apply(classify_risk)
```

3.3 Machine Learning: K-Means Clustering

Service Desert Identification

```
from sklearn.cluster import KMeans  
from sklearn.preprocessing import StandardScaler  
  
# Prepare features for clustering  
cluster_features = df_gap_analysis[['mbu_eligible', 'bio_age_5_17', 'mbu_gap',  
'mbu_compliance_ratio']]  
  
# Standardize features  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(cluster_features)  
  
# Optimal clusters via Elbow Method + Silhouette Score  
kmeans = KMeans(n_clusters=4, random_state=42)  
clusters = kmeans.fit_predict(X_scaled)
```

3.4 Deep Learning: LSTM Time-Series Forecaster

LSTM Architecture

```
class MBU_LSTM(nn.Module):  
    def __init__(self, input_size=1, hidden_size=64, num_layers=2, dropout=0.2):  
        super(MBU_LSTM, self).__init__()  
  
        self.lstm = nn.LSTM(  
            input_size=input_size,  
            hidden_size=hidden_size,  
            num_layers=num_layers,  
            batch_first=True,  
            dropout=dropout if num_layers > 1 else 0  
        )  
  
        self.fc = nn.Sequential(  
            nn.Linear(hidden_size, 32),
```

```

        nn.ReLU(),
        nn.Dropout(dropout),
        nn.Linear(32, 1)
    )

    def forward(self, x):
        h0 = torch.zeros(self.num_layers, x.size(0),
self.hidden_size).to(x.device)
        c0 = torch.zeros(self.num_layers, x.size(0),
self.hidden_size).to(x.device)
        out, (hn, cn) = self.lstm(x, (h0, c0))
        out = out[:, -1, :]
        out = self.fc(out)
        return out

```

Training Configuration

- **Optimizer:** Adam (lr=0.001)
- **Loss Function:** MSE (Mean Squared Error)
- **Epochs:** 100
- **Learning Rate Scheduler:** ReduceLROnPlateau

3.5 Autoencoder Anomaly Detection

```

class DistrictAutoencoder(nn.Module):
    def __init__(self, input_dim):
        super(DistrictAutoencoder, self).__init__()

        # Encoder: Compress to latent space
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, 8) # Bottleneck
        )

        # Decoder: Reconstruct from latent space
        self.decoder = nn.Sequential(
            nn.Linear(8, 16),
            nn.ReLU(),
            nn.Linear(16, 32),
            nn.ReLU(),
            nn.Linear(32, input_dim)
        )

        # Anomaly Threshold: Mean + 2*Std of reconstruction error
        threshold = np.mean(reconstruction_errors) + 2 * np.std(reconstruction_errors)

```

3.6 Explainable AI with SHAP

```
import shap

# Wrap K-Means for SHAP
def kmeans_predict_proba(X):
    distances = kmeans.transform(X)
    exp_neg_dist = np.exp(-distances)
    probs = exp_neg_dist / exp_neg_dist.sum(axis=1, keepdims=True)
    return probs

# Calculate SHAP values
explainer = shap.KernelExplainer(kmeans_predict_proba, background)
shap_values = explainer.shap_values(X_explain)
```

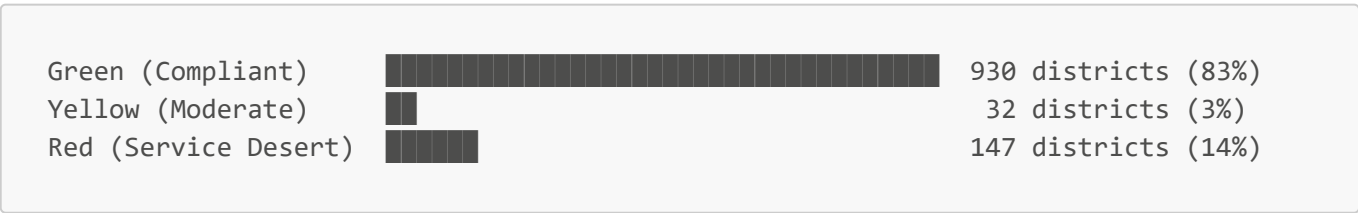
4. Data Analysis and Visualisation

4.1 Key Findings

Summary Statistics

Metric	Value
Total Districts Analyzed	1,109
MBU Eligible Children	5,226,855
Children at Risk (MBU Gap)	177,900
Service Desert Districts	147 (14%)
Moderate Risk Districts	32 (3%)
Compliant Districts	930 (83%)

Risk Distribution



Financial Impact

Category	Amount at Risk
Scholarships (Rs. 18,000/child)	Rs. 32.02 Crore

Category	Amount at Risk
DBT Benefits (Rs. 6,000/child)	Rs. 10.67 Crore
Total Impact	Rs. 42.69 Crore

4.2 Visualizations

Chart 1: Districts Requiring Attention by State

This horizontal bar chart shows the number of problem districts (Service Desert + Moderate Risk) per state, with color indicating the severity.

Key Insight: Meghalaya leads with 25 problem districts, followed by Orissa and Rajasthan.

Chart 2: MBU Gap by State - Children at Risk

A bar chart showing the number of children at risk of losing benefits in each state.

Top 5 States by MBU Gap:

- 1. Meghalaya: 37,953 children
- 2. Bihar: 36,170 children
- 3. Karnataka: 24,321 children
- 4. Haryana: 9,949 children
- 5. Andhra Pradesh: 3,894 children

Chart 3: Service Desert Identification - District Clustering

A scatter plot showing district clustering based on:

- X-axis: MBU Eligible Population
- Y-axis: Compliance Ratio (%)
- Size: MBU Gap (children at risk)
- Color: Risk Category

Thresholds Marked:

-  Safe Threshold: 80% compliance
-  Critical Threshold: 50% compliance

Chart 4: Daily Biometric Update Trends

A time-series chart showing:

- Blue line: Total biometric updates per day
- Red dashed line: 7-day moving average

Key Observation: Sharp drop in updates around September 2025.

Chart 5: LSTM Forecast - MBU Demand Prediction

A chart comparing:

- Blue: Actual historical updates
- Green: LSTM model predictions
- Orange: 6-month forecast

School Admission Rush (June-July 2026) highlighted showing predicted demand of ~7M updates/month.

Chart 6: Model Benchmarking

Comparison of LSTM vs traditional methods:

Model	RMSE	MAE	MAPE
LSTM (Deep Learning)	1,710,108	1,401,974	20.5%
Linear Regression	1,739,611	1,429,372	20.3%
Moving Average	2,054,553	1,706,931	26.6%

Result: LSTM reduces error by 1.7% vs Linear Regression and 17% vs Moving Average.

Chart 7: Autoencoder Anomaly Detection

A histogram showing reconstruction error distribution:

- Green bars: Normal districts
- Red bars: Anomalous districts (above threshold)

21 districts flagged as anomalous for further investigation.

Chart 8: SHAP Feature Importance

A horizontal bar chart showing which features drive Service Desert classification:

1. **Compliance Ratio:** 0.194 (highest impact)
2. **Bio Updates:** 0.175
3. **MBU Gap:** 0.062
4. **MBU Eligible:** 0.012

Chart 9: Interactive Geospatial Map

A Folium map of India showing Service Desert locations:

-  Red markers: States with 5+ Service Desert districts (CRITICAL)
-  Orange markers: States with 1-4 Service Desert districts (AT RISK)
-  Green markers: States with 0 Service Desert districts (COMPLIANT)

Popups show: State name, status, Service Deserts count, MBU Gap, total districts.

4.3 Deep Learning Module Results

Module	Output
--------	--------

Module	Output
LSTM Forecaster	6-month demand forecast predicting ~7M updates/month
Autoencoder	21 anomalous districts detected
SHAP Explainability	Compliance Ratio is strongest predictor
Folium Map	Interactive map saved to <code>service_desert_map.html</code>

5. Code Files

5.1 Repository Structure

```
UIDAI_DH_2k26/
├── MBU_Gap_Analyzer.ipynb           # Main analysis notebook (ML)
├── MBU_Gap_Analyzer_DeepLearning.ipynb # Deep Learning enhanced notebook
├── requirements.txt                 # Python dependencies
├── README.md                       # Project documentation
├── PRESENTATION.md                 # Presentation notes
├── analysis_outputs/               # Generated outputs
│   ├── service_desert_map.html     # Interactive Folium map
│   └── *.csv                       # Analysis CSV exports
└── uidai_datasets/                 # UIDAI provided data
    ├── api_data_aadhar_enrolment/
    ├── api_data_aadhar_biometric/
    └── api_data_aadhar_demographic/
```

5.2 GitHub Repository

URL: https://github.com/STIWARTs/UIDAI_DH_2k26

Branches:

- `MAIN`: Complete solution with Deep Learning modules
- `deep-learning`: Development branch (merged)

5.3 Key Code Sections

The complete code is available in the Jupyter notebooks. Key sections include:

1. **Data Loading** (Cell 4-5): Load and concatenate CSV files
2. **Preprocessing** (Cell 6-8): Date parsing, aggregation
3. **Gap Analysis** (Cell 9-11): MBU gap calculation, risk classification
4. **K-Means Clustering** (Cell 12-15): Service Desert identification
5. **LSTM Forecasting** (Cell 16-22): Deep learning time-series prediction
6. **Model Benchmarking** (Cell 23): LSTM vs Linear Regression vs Moving Average
7. **Autoencoder** (Cell 24-28): Anomaly detection

- 8. **SHAP Analysis** (Cell 29-32): Feature importance explanation
- 9. **Folium Map** (Cell 33-36): Interactive geospatial visualization

Policy Recommendations

Based on our analysis, we recommend:

#	Recommendation	Target	Impact
1	Deploy Mobile Aadhaar Seva Kendras	147 Service Desert districts	Reach underserved areas
2	Launch SMS/IVR Reminder Campaigns	Children turning 5/15 years	Proactive awareness
3	Integrate MBU Status Check	School admission portals	Early flagging
4	Extend Fee Waiver	Low-compliance states	Remove financial barrier
5	Deploy Real-time Dashboard	District Collectors	Weekly monitoring

Tech Stack

Category	Technologies
Core	Python 3.10+, Pandas, NumPy
ML	Scikit-Learn (K-Means, StandardScaler)
Deep Learning	PyTorch (LSTM, Autoencoder)
Explainability	SHAP
Visualization	Plotly, Folium, Matplotlib
Time Series	Prophet (optional)

Submitted by: Team OMEGA
UIDAI Data Hackathon 2026

Built with  for Digital India
Making Aadhaar work for every child